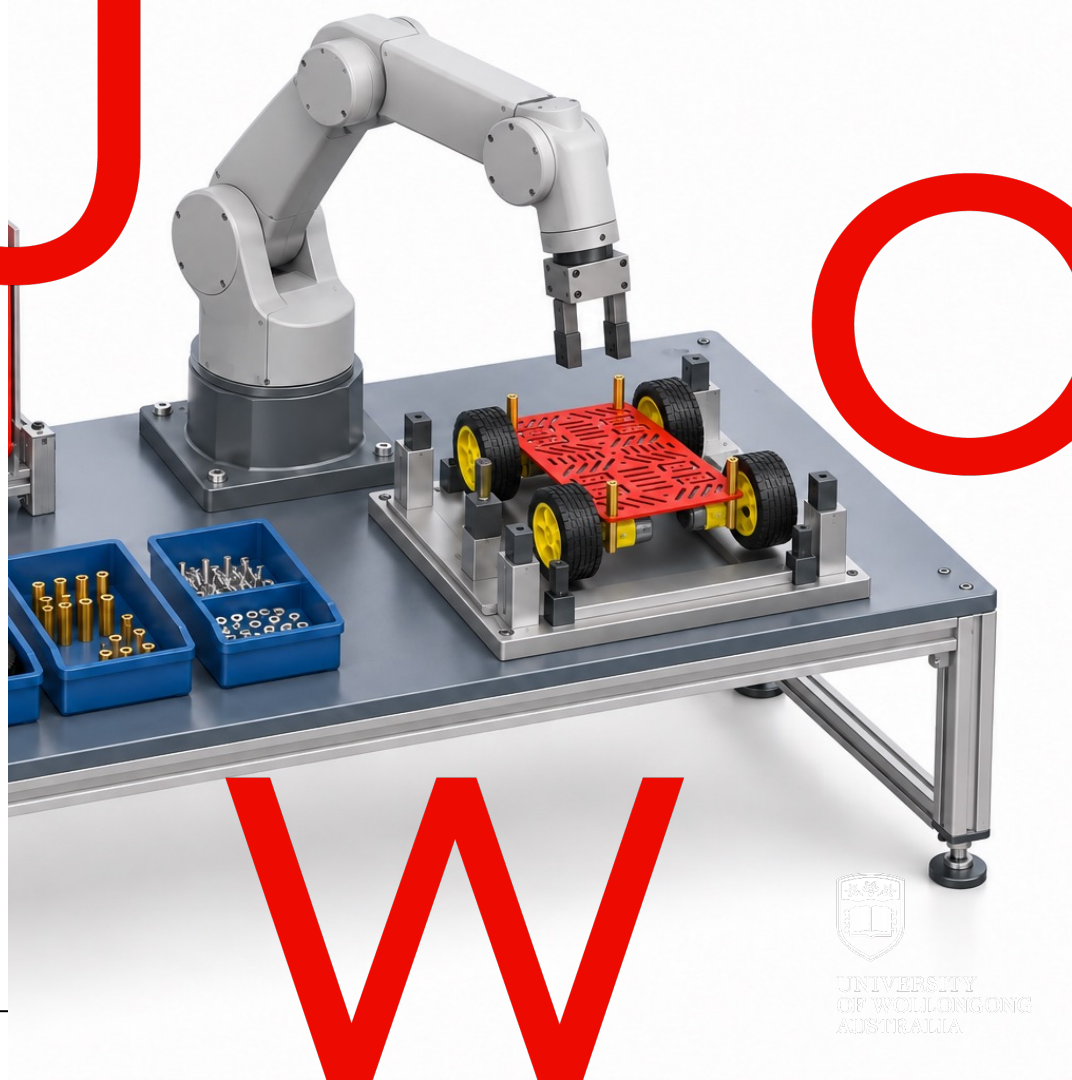
Robotics and Flexible Automation

Lecture 2

3D representation



Prof. Zengxi (Stephen) Pan

BJTU · 2026

Table of Contents

1 Coordinate frames and rotation matrices

2 Orientation representations and homogeneous transformations

3 Frame chains and project application

PART 1

Coordinate frames and rotation matrices

From physical frames to a valid 3D orientation

1 Coordinate frames and pose Slides 4–10

2 Rotation matrices Slides 11–16

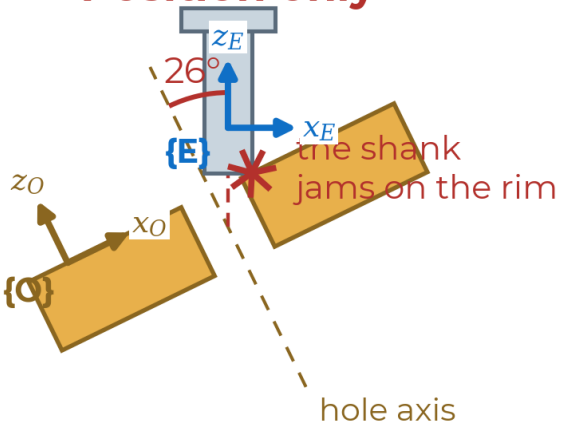
3 Composite rotations Slides 17–20

Orientation in an insertion task

CORE

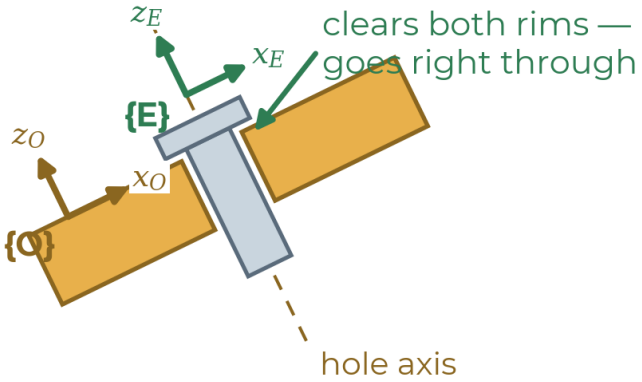
Generic pin-in-hole orientation example

Position only



Over the hole, but pointing the wrong way.
The pin cannot enter.

Position AND orientation



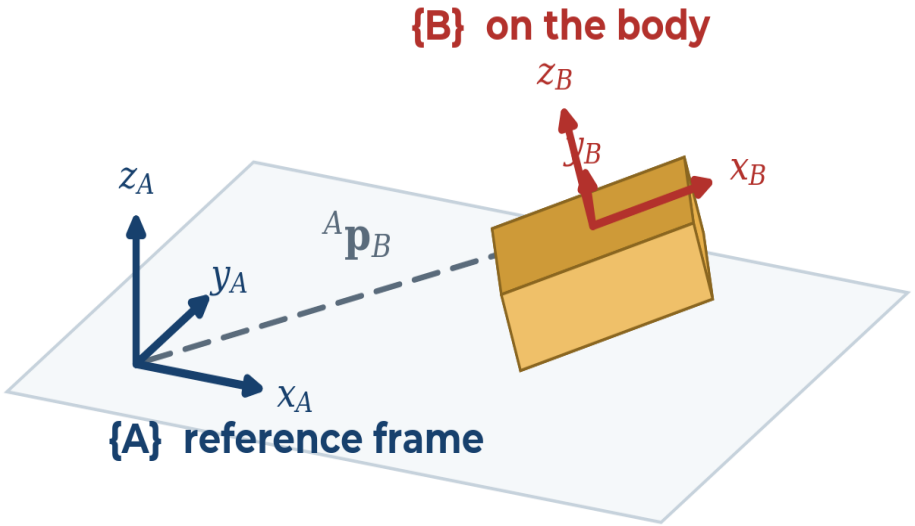
The pin axis and the hole axis coincide.
This is a usable robot target.

A robot target is FULL definitions of where everything is.
(3 dimensional world: 6 DoFs)

Coordinate frames

CORE

A frame is attached to something physical, and described relative to another frame



Notation used in this course

- **{A}, {B}, {C}** named coordinate frames
- **p** = position; **R** = orientation; **T** = full pose
- ${}^A\mathbf{p}$ point P expressed in frame {A}
- ${}^A\mathbf{p}_B$ origin OB expressed in {A} (dashed arrow)
- ${}^A\mathbf{R}_B$ orientation of {B} relative to {A}
- ${}^A\mathbf{T}_B$ full pose of {B} relative to {A}

The dashed arrow shows the translation ${}^A p_B$ from O_A to O_B.

Position + orientation = pose

CORE

And the standard way to carry both at once is one 4×4 matrix

POSITION 3 × 1

Where the origin of {B} sits, measured in {A}.

ORIENTATION 3 × 3

Which way {B} is turned. The columns are the axes of {B}, written in {A}.

POSE 4 × 4

Both together — six degrees of freedom in one matrix.

$$ {}^{A}p = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} \quad + \quad {}^{A}R_B = [n\ t\ b] \quad = \quad {}^{A}T_B = \begin{bmatrix} {}^{A}R_B & {}^{A}p_B \\ 0 & 1 \end{bmatrix} $$

$$ = \begin{bmatrix} n_x & t_x & b_x \\ n_y & t_y & b_y \\ n_z & t_z & b_z \end{bmatrix} $$

T is the default representation in this course. Euler angles, roll–pitch–yaw, axis–angle and quaternions are other ways of representation of orientation.

Position and orientation in 3D

CORE

Position is easy. Orientation is the rest of this lecture.

Position in 3D

— AP = x x̂ + y ŷ + z ẑ — three numbers, unambiguous once the frame {A} is named.

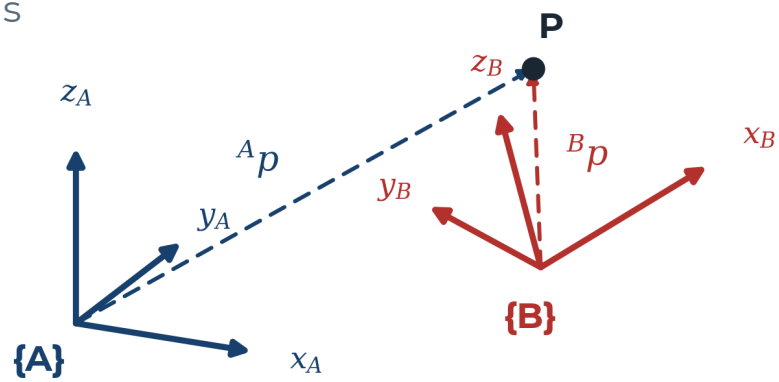
Orientation in 3D

— **Orthonormal rotation matrix**

— Three-angle representations — Euler, RPY

— Axis and angle

— Unit quaternions



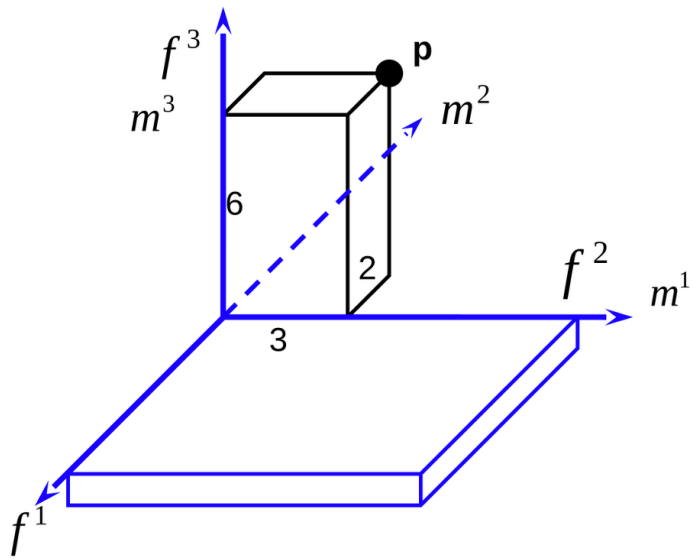
Orthonormal frames: example

CORE

The same point, two frames, two sets of numbers

Point ***p***, as a column vector in each frame:

$$ {}^{F}p = {}^{F}R_{M}\,{}^{M}p = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ 2 \\ 6 \end{bmatrix} = \begin{bmatrix} -2 \\ 3 \\ 6 \end{bmatrix}. $$



$$ {}^{F}p = \begin{bmatrix} -2 \\ 3 \\ 6 \end{bmatrix} \quad {}^{M}p = \begin{bmatrix} 3 \\ 2 \\ 6 \end{bmatrix} $$

Same point, same origin, different numbers. What converts one column into the other is the rotation matrix.

Coordinate transformation: common origin

CORE

Definition of the rotation matrix R

Let F and M share an origin, and let R hold the dot products of their axes:

$$^{F}p = R^{M}p \qquad R = \begin{bmatrix} f_1 \cdot m_1 & f_1 \cdot m_2 & f_1 \cdot m_3 \\ f_2 \cdot m_1 & f_2 \cdot m_2 & f_2 \cdot m_3 \\ f_3 \cdot m_1 & f_3 \cdot m_2 & f_3 \cdot m_3 \end{bmatrix}$$

f1: unit vector at f1 direction represented in m frame

$$R = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

m1: unit vector at m1 direction represented in f frame

Columns

Column j is the j-th axis of M, in F.

Rows

Row k is the k-th axis of F, in M.

If you can read a rotation matrix as three unit vectors, you can write one down directly from a sketch — no trigonometry required.

Orientation of the tool

CORE

The three columns have names: normal, tangent (sliding) and binormal (approach)

The tool orientation is the tool-frame axes, written in the base frame.

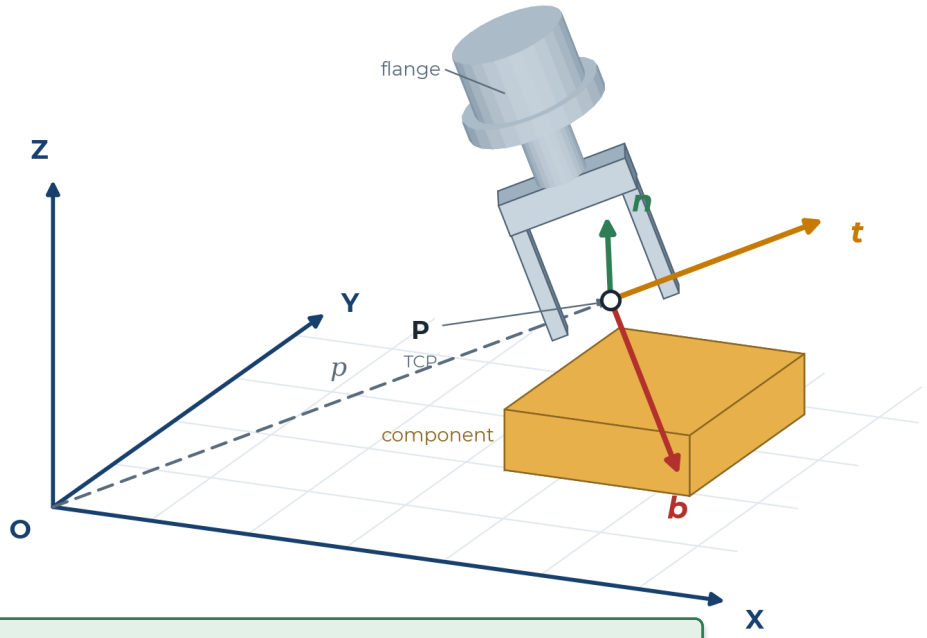
— n normal

— t tangent / sliding

— b binormal / approach

b is the one your project cares about — the direction the tool comes in along.

$$ {}^{A}R_{B} = [n\ t\ b] = \begin{bmatrix} n_x & t_x & b_x \\ n_y & t_y & b_y \\ n_z & t_z & b_z \end{bmatrix} $$



Set tool axes explicitly; L4 uses another jaw-axis convention.

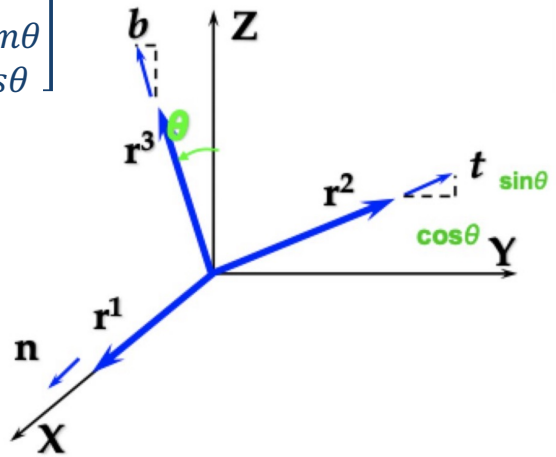
The three fundamental rotations

CORE

Read the columns straight off the diagram — no derivation needed

$$\mathbf{n} = \begin{bmatrix}1\\0\\0\end{bmatrix}, \quad \mathbf{t} = \begin{bmatrix}0\\cos\theta\\sin\theta\end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix}0\\-sin\theta\\cos\theta\end{bmatrix}$$

$$R_x(\theta) = \begin{bmatrix}1 & 0 & 0\\0 & cos\theta & -sin\theta\\0 & sin\theta & cos\theta\end{bmatrix}$$



$$R_y(\theta) = \begin{bmatrix}cos\theta & 0 & sin\theta\\0 & 1 & 0\\-sin\theta & 0 & cos\theta\end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix}cos\theta & -sin\theta & 0\\sin\theta & cos\theta & 0\\0 & 0 & 1\end{bmatrix}$$

How to read them

Each column is a rotated unit vector; the rest follows from the cyclic order x → y → z → x.

The one that catches people

Ry carries its minus sign bottom-left, not top-right — right-handedness, not a typo.

Trigonometric shorthand

OPTIONAL

Kinematic expressions grow fast. This shorthand keeps them readable.

Symbol	Description
$S\theta$	$\sin\theta$
$C\theta$	$\cos\theta$
S_k	$\sin\theta_k$
C_k	$\cos\theta_k$
S_{kj}	$\sin(\theta_k+\theta_j)$
C_{kj}	$\cos(\theta_k+\theta_j)$
S_{k-j}	$\sin(\theta_k-\theta_j)$
C_{k-j}	$\cos(\theta_k-\theta_j)$
atan2 (y, x)	four-quadrant arctan (y/x)
$V\theta$	$1-\cos\theta$

atan2 matters.

Ordinary arctan loses the quadrant and returns the wrong branch for half of all inputs.

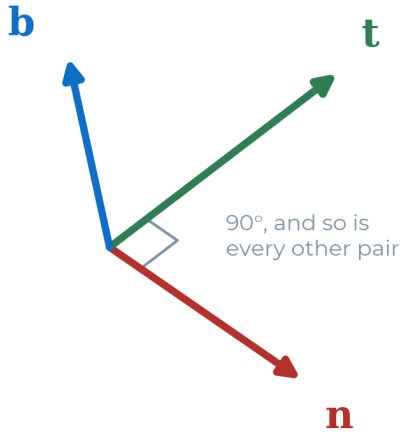
Every inverse-kinematics solution in L4 uses atan2 for exactly that reason.

A rotation matrix is not nine free numbers

CORE

Six constraints on the three columns leave only three freedoms

$$R = [n\ t\ b] = \begin{bmatrix} n_x & t_x & b_x \\ n_y & t_y & b_y \\ n_z & t_z & b_z \end{bmatrix}$$ *the tool axes, in the reference frame*



$$\|n\| = \|t\| = \|b\| = 1 \qquad unit\ length$$

$$n \cdot t = t \cdot b = b \cdot n = 0 \qquad perpendicular$$

$$n \times t = b \qquad right-handed$$

9 entries the matrix R	–	**3** unit length	–	**3** perpendicular	=	**3** free

Three independent parameters — the same three orientation freedoms as the six-DOF count in L01

Two important properties

CORE

Orthonormality, written as one matrix equation

$$R^T R = \begin{bmatrix} n\cdot n & n\cdot t & n\cdot b \\ t\cdot n & t\cdot t & t\cdot b \\ b\cdot n & b\cdot t & b\cdot b \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = I$$

$$R^{-1} = R^T \qquad detR = 1$$

Inverting an orientation costs one transpose — no division and no determinant. That is why controllers store orientation as a matrix even though it is nine numbers.

det R = +1, not −1

RTR = I with det R = −1 is a reflection — a left-handed frame. No robot can do that.

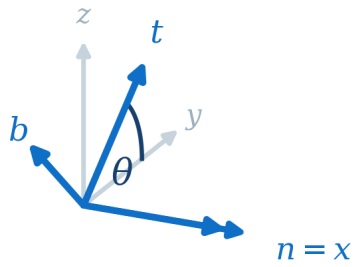
Watch for drift

Repeated products slowly drift away from RTR = I. Every library has a normalise function for it.

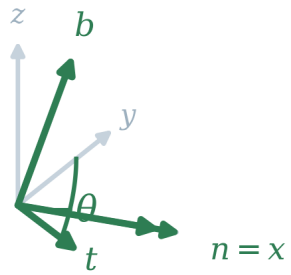
Inverse rotation

CORE

Rotating back is the negative angle — and that is the same as the transpose



rotate the frame by $+\theta$ about x



rotate it back by $-\theta$

Physically: undo it by turning −θ about the same axis.

Algebraically: cos is even and sin is odd, so negating θ swaps the off-diagonal signs — a transpose.

$$ {}^{F}P = R\,{}^{M}P \quad \Leftrightarrow \quad {}^{M}P = R^{T}\,{}^{F}P $$

$$ R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\theta & -s_\theta \\ 0 & s_\theta & c_\theta \end{bmatrix} \qquad R_x(-\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\theta & s_\theta \\ 0 & -s_\theta & c_\theta \end{bmatrix} = R_x(\theta)^T $$

Both directions, free

One matrix converts coordinates both ways — never a matrix inverse.

SO(3): the modern name for what we just built

OPTIONAL

You will meet this notation in every current textbook, paper and library

$$SO(3) = \{R \in \mathbb{R}^{3\times3} : R^T R = I, \quad detR = 1\}$$

What it says

The set of all valid 3D rotation matrices.

"Special" — determinant +1, so a rotation and not a reflection.

"Orthogonal" — RTR = I, so the columns stay orthonormal.

Why it is worth naming

It tells you what is NOT a rotation.

Numerical drift pushes a matrix out of SO(3); software renormalises it.

It is the vocabulary of L5, L8 and every robotics library.

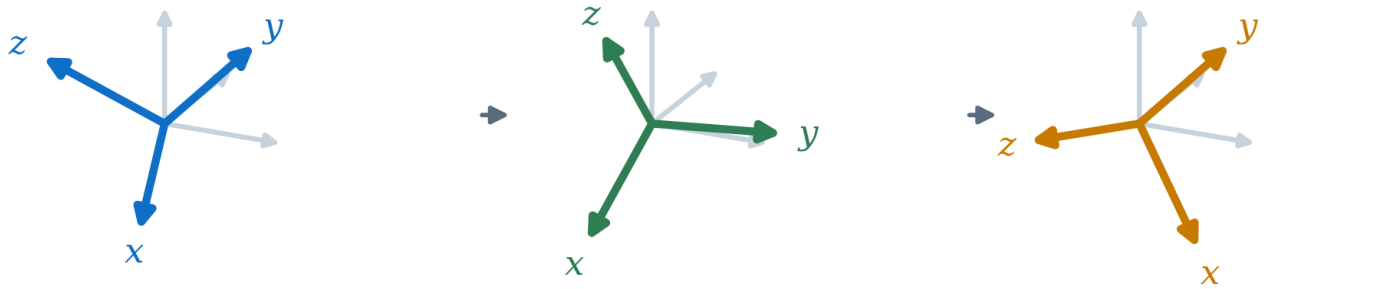
No group theory required. Recognise the notation, know the two conditions, move on.

Composite rotations

CORE

A second rotation about a fixed axis, or about the frame's own axis, gives different results

grey = reference frame *same R_k, two different results*



start: R **about a FIXED axis: $R_k R$** **about its OWN axis: $R R_k$**

$$fixed\ axis: \quad R \leftarrow R_k(\theta)R \qquad own\ axis: \quad R \leftarrow RR_k(\theta)$$

Fixed axis → pre-multiply. Own axis → post-multiply. That single rule is the whole of composite rotation.

Composite rotations

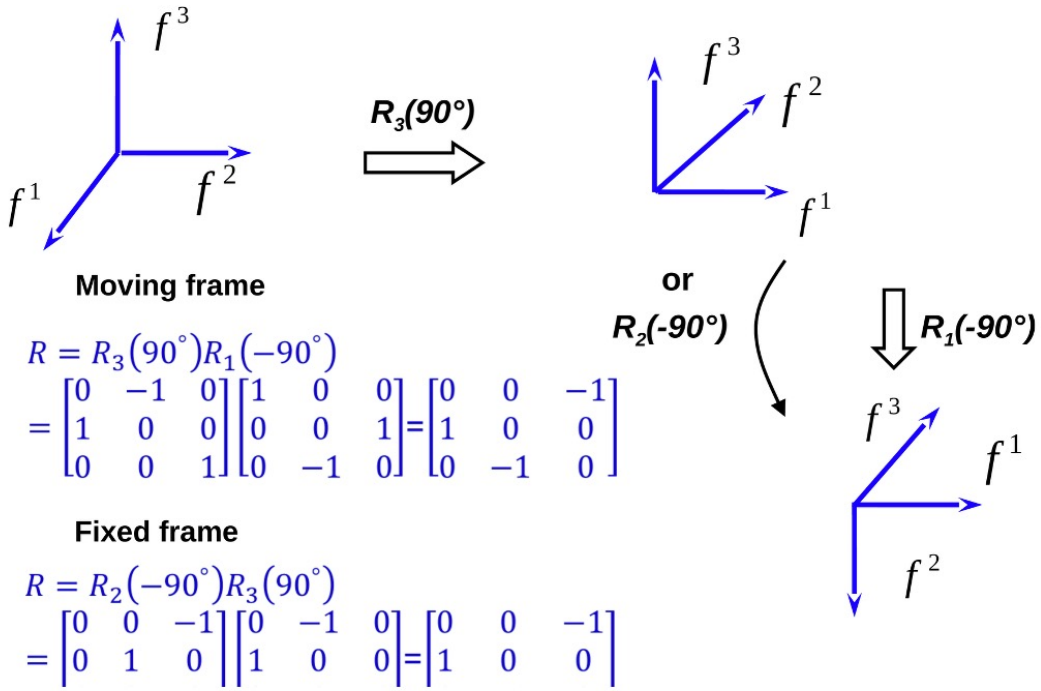
moving-frame vs. fixed-frame

Fixed frame:

$$R \leftarrow R_k(\theta)R$$

Moving frame:

$$R \leftarrow RR_k(\theta)$$



Moving frame: R = $R_3(90°)\,R_1(-90°)$. Fixed frame: R = $R_2(-90°)\,R_3(90°)$. Same final orientation, reached by different sequences.

MATLAB check: Corke Spatial Math

OPTIONAL

RTB10 environment; rotations in radians

```
>> R = rotx(pi/2)
>> trplot(R)
>> tranimate(R)
```

```
>> R1 = rotx(pi/2) * roty(pi/2)
>> R2 = roty(pi/2) * rotx(pi/2)

>> isequal(R1, R2)      %  ans = 0
```

Use tranimate

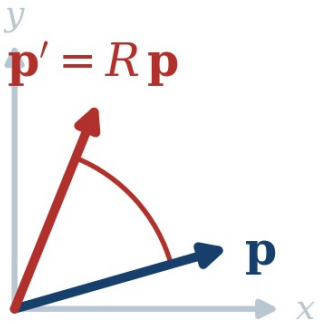
Animating the rotation is the fastest way to see which axis a product actually turns about.

Run both orders and watch the difference. Ten seconds of animation replaces a page of algebra.

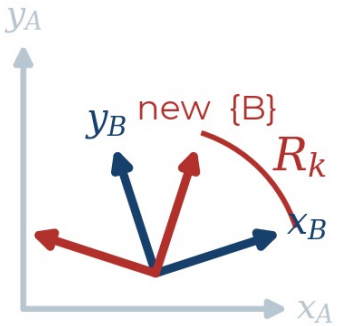
You will need this habit in L5.

Three things a rotation matrix is used for

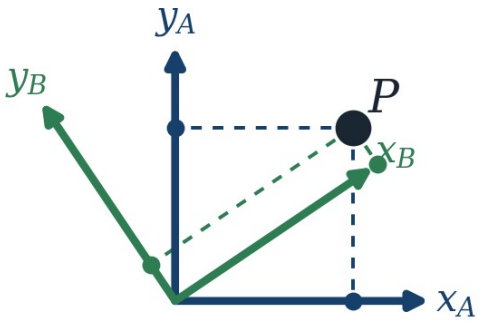
CORE



Rotate a vector
one frame;
the object moves



Re-orient a frame
the frame turns;
the rule picks the side



Re-express a point
the point stays put;
its numbers change

$$\mathbf{p}' = R\mathbf{p} \qquad R \leftarrow R_k R \text{ or } R R_k \qquad {}^{A}\mathbf{p} = {}^{A}R_B\,{}^{B}\mathbf{p}$$

PART 2

Orientation representations and transformations

Alternative orientation parameters and complete 3D poses

1 Three-angle representations Slides 22–26

2 Homogeneous transformations Slides 27–34

3 Axis–angle and quaternions Slides 35–41

Three angles instead of nine numbers

CORE

More ways to represent orientation:
Any orientation is three successive rotations about coordinate axes

Nine entries, three freedoms — so a three-number description exists.

Twelve sequences are valid: six proper Euler and six Tait–Bryan.

"Euler angles" alone is not a specification.

Proper Euler angles	Tait–Bryan angles
$X_1Z_2X_3 = \begin{bmatrix} c_2 & -c_3s_2 & s_2s_3 \\ c_1s_2 & c_1c_2c_3 - s_1s_3 & -c_3s_1 - c_1c_2s_3 \\ s_1s_2 & c_1s_3 + c_2c_3s_1 & c_1c_3 - c_2s_1s_3 \end{bmatrix}$	$X_1Z_2Y_3 = \begin{bmatrix} c_2c_3 & -s_2 & c_2s_3 \\ s_1s_3 + c_1c_3s_2 & c_1c_2 & c_1s_2s_3 - c_3s_1 \\ c_3s_1s_2 - c_1s_3 & c_2s_1 & c_1c_3 + s_1s_2s_3 \end{bmatrix}$
$X_1Y_2X_3 = \begin{bmatrix} c_2 & s_2s_3 & c_3s_2 \\ s_1s_2 & c_1c_3 - c_2s_1s_3 & -c_1s_3 - c_2c_3s_1 \\ -c_1s_2 & c_3s_1 + c_1c_2s_3 & c_1c_2c_3 - s_1s_3 \end{bmatrix}$	$X_1Y_2Z_3 = \begin{bmatrix} c_2c_3 & -c_2s_3 & s_2 \\ c_1s_3 + c_3s_1s_2 & c_1c_3 - s_1s_2s_3 & -c_2s_1 \\ s_1s_3 - c_1c_3s_2 & c_3s_1 + c_1s_2s_3 & c_1c_2 \end{bmatrix}$
$Y_1X_2Y_3 = \begin{bmatrix} c_1c_3 - c_2s_1s_3 & s_1s_2 & c_1s_3 + c_2c_3s_1 \\ s_2s_3 & c_2 & -c_3s_2 \\ -c_3s_1 - c_1c_2s_3 & c_1s_2 & c_1c_2c_3 - s_1s_3 \end{bmatrix}$	$Y_1X_2Z_3 = \begin{bmatrix} c_1c_3 + s_1s_2s_3 & c_3s_1s_2 - c_1s_3 & c_2s_1 \\ c_2s_3 & c_2c_3 & -s_2 \\ c_1s_2s_3 - c_3s_1 & c_1c_3s_2 + s_1s_3 & c_1c_2 \end{bmatrix}$
$Y_1Z_2Y_3 = \begin{bmatrix} c_1c_2c_3 - s_1s_3 & -c_1s_2 & c_3s_1 + c_1c_2s_3 \\ c_3s_2 & c_2 & s_2s_3 \\ -c_1s_3 - c_2c_3s_1 & s_1s_2 & c_1c_3 - c_2s_1s_3 \end{bmatrix}$	$Y_1Z_2X_3 = \begin{bmatrix} c_1c_2 & s_1s_3 - c_1c_3s_2 & c_3s_1 + c_1s_2s_3 \\ s_2 & c_2c_3 & -c_2s_3 \\ -c_2s_1 & c_1s_3 + c_3s_1s_2 & c_1c_3 - s_1s_2s_3 \end{bmatrix}$
$Z_1Y_2Z_3 = \begin{bmatrix} c_1c_2c_3 - s_1s_3 & -c_3s_1 - c_1c_2s_3 & c_1s_2 \\ c_1s_3 + c_2c_3s_1 & c_1c_3 - c_2s_1s_3 & s_1s_2 \\ -c_3s_2 & s_2s_3 & c_2 \end{bmatrix}$	$Z_1Y_2X_3 = \begin{bmatrix} c_1c_2 & c_1s_2s_3 - c_3s_1 & s_1s_3 + c_1c_3s_2 \\ c_2s_1 & c_1c_3 + s_1s_2s_3 & c_3s_1s_2 - c_1s_3 \\ -s_2 & c_2s_3 & c_2c_3 \end{bmatrix}$
$Z_1X_2Z_3 = \begin{bmatrix} c_1c_3 - c_2s_1s_3 & -c_1s_3 - c_2c_3s_1 & s_1s_2 \\ c_3s_1 + c_1c_2s_3 & c_1c_2c_3 - s_1s_3 & -c_1s_2 \\ s_2s_3 & c_3s_2 & c_2 \end{bmatrix}$	$Z_1X_2Y_3 = \begin{bmatrix} c_1c_3 - s_1s_2s_3 & -c_2s_1 & c_1s_3 + c_3s_1s_2 \\ c_3s_1 + c_1s_2s_3 & c_1c_2 & s_1s_3 - c_1c_3s_2 \\ -c_2s_3 & s_2 & c_2c_3 \end{bmatrix}$

A triple of angles is meaningless until you state the axis order and whether the axes are fixed or moving. State both, every time.

X–Y–Z Euler angles

CORE

The convention used for the rest of this course

$$R = R_x(\alpha)R_y(\beta)R_z(\gamma)$$

$$R = \begin{bmatrix} c_\beta c_\gamma & -c_\beta s_\gamma & s_\beta \\ c_\alpha s_\gamma + s_\alpha s_\beta c_\gamma & c_\alpha c_\gamma - s_\alpha s_\beta s_\gamma & -s_\alpha c_\beta \\ s_\alpha s_\gamma - c_\alpha s_\beta c_\gamma & s_\alpha c_\gamma + c_\alpha s_\beta s_\gamma & c_\alpha c_\beta \end{bmatrix}$$

Every sequence has a partner

X–Y–Z about moving axes gives the same matrix as Z–Y–X about fixed axes.

One orientation, two descriptions — so the convention must be written down.

MOVING axes:

- x by α, then new y by β, then new z by γ.

FIXED axes:

- z by γ, then y by β, then x by α.

R = rotx(a) * roty(b) * rotz(g) % RTB, radians

Three angles are not unique

CORE

The orientation is unique. Its parameterisation is not.

For X–Y–Z, two different triples always give the same R:

$$(\alpha, \beta, \gamma) = (0.3, 0.4, 0.5) \qquad (\alpha - \pi, \pi - \beta, \gamma - \pi)$$

$$= (-2.8416, 2.7416, -2.6416) \quad give\ the\ same\ R$$

The middle angle reflects about π/2, and the outer two shift by π.
An inverse conversion returns one branch, which may differ from the input.
At β = ±π/2, only a combination of the outer angles remains. This is gimbal lock.

What to do about it

Never compare two orientations by comparing angle triples — compare R, or compare quaternions.

Store and interpolate orientation as R or as a quaternion. Use three angles for display and for typing into a teach pendant.

Roll, pitch and yaw

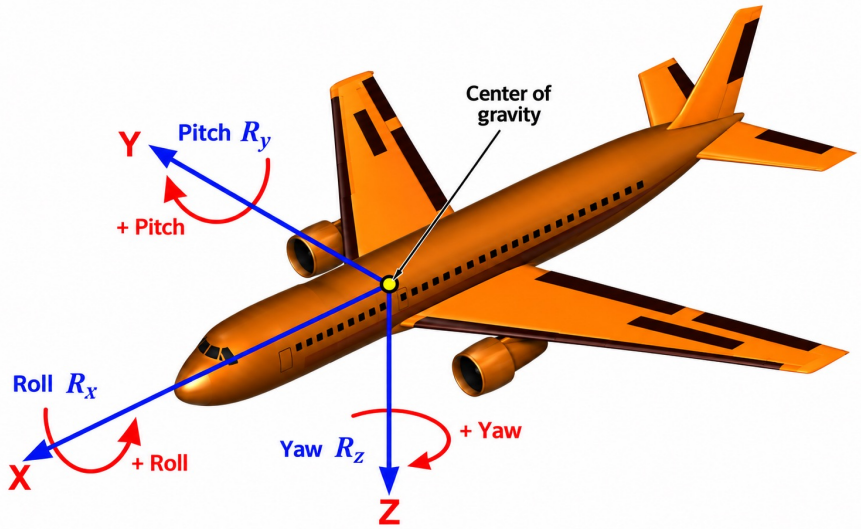
CORE

roll about X, pitch about Y, yaw about Z
(the aviation names)

RPY is X–Y–Z about the FIXED frame:

$$R = R_z(\psi)R_y(\theta)R_x(\phi)$$

φ = roll, θ = pitch, ψ = yaw.
Fixed-axis rotations pre-multiply.



X–Y–Z fixed angles ≡ Z–Y–X moving angles. Same matrix, two descriptions.

Orientation in real robot software

OPTIONAL

The same physical pose, written four different ways by four vendors

System	How orientation is expressed	What you must check
ABB · RAPID	Unit quaternion in a robtarget	q and −q are the same pose
KUKA · KRL	A, B, C angles — Z–Y–X order	Which axis each letter means
FANUC · TP	W, P, R angles — X–Y–Z order	Fixed or moving axes
ROS · tf2	Quaternion, with RPY helpers	Frame naming and parent/child order
MATLAB · Corke RTB10	rpy2r, eul2tr, tr2rpy …	RADIANS unless you pass 'deg'

This is why the lecture insists on rotation matrices as the common ground: every one of these converts to and from R without ambiguity.

The homogeneous transformation matrix

CORE

Append a 1 to every point, and a pose chain becomes a matrix product

$$ {}^{A}T_{B} = \begin{bmatrix} {}^{A}R_{B} & {}^{A}p_{B} \\ 0 \quad 0 \quad 0 & 1 \end{bmatrix} = \begin{bmatrix} n_x & t_x & b_x & P_x \\ n_y & t_y & b_y & P_y \\ n_z & t_z & b_z & P_z \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

R P

ARB top-left 3×3

Orientation of {B} in {A} — columns n, t, b.

ApB top-right 3×1

Position of the origin of {B}, in {A}.

0 0 0 bottom-left

Always zero in robotics.

1 bottom-right

A scale factor, always 1 here.

Mapping a point between frames

$$ {}^{A}p = {}^{A}T_{B}\,{}^{B}p $$

Take a point in {B}, append a 1, multiply by ATB — and you have the same point in {A}.

From base to tool: Composite homogeneous transformations CORE

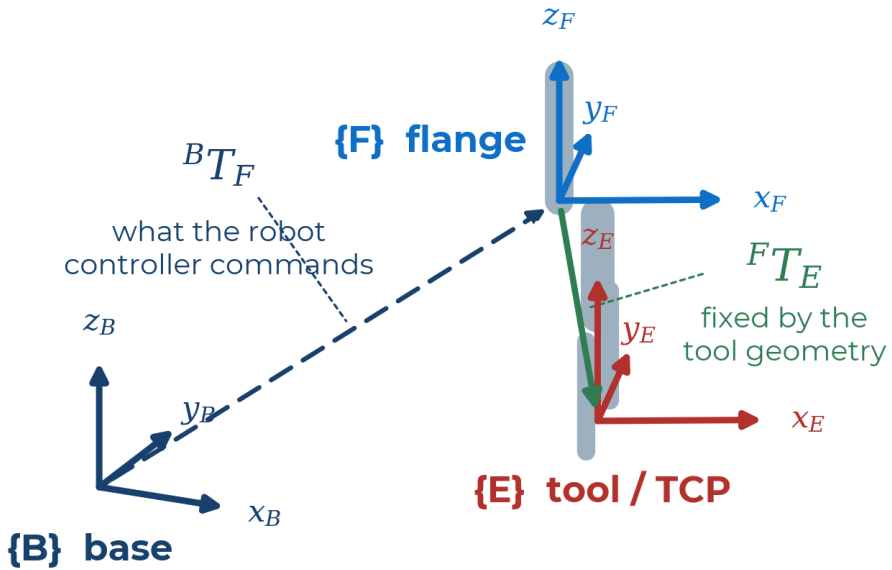
The order matters

{F} is the flange at the end of the wrist. {E} is the tool centre point.

FTE is fixed by the tool you bolt on.

This is the "tooldata" on a teach pendant. Get it wrong and the robot is repeatable but consistently in the wrong place.

BTF is forward kinematics.



$$^BT_E = {}^BT_F\,{}^FT_E \quad \Leftrightarrow \quad {}^BT_F = {}^BT_E({}^FT_E)^{-1}$$

Worked example: a homogeneous translation

WORKED EXAMPLE

Translate M by 5 along f^1 and −3 along f^2, and find the new coordinates

Given: $[q]^M = [0\ \ 0\ \ 10\ \ 1]^T$

Translate M by 5 units along f^1 and by −3 units along f^2.

Find the homogeneous coordinates of q after the translation.

$$[q]^F = tran(5,-3,0)[q]^M = \begin{bmatrix} 1 & 0 & 0 & 5 \\ 0 & 1 & 0 & -3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 10 \\ 1 \end{bmatrix} = \begin{bmatrix} 5 \\ -3 \\ 10 \\ 1 \end{bmatrix}$$

$$^A p = {}^A T_B \, {}^B p$$

Answer: $[q]^F = [5\ \ -3\ \ 10\ \ 1]^T$

Note that the z coordinate is untouched, and the trailing 1 survives — two quick checks that the arithmetic is right.

Worked example: translate, then rotate

Translate M1 along f2 by 3 units, then rotate M1 about f3 by π

Both operations are about axes of the FIXED frame F.

So both are pre-multiplied, and the matrix written on the left is the one applied last:

$[m^1]^F$ = rotz(π) · trany(3) · $[m^1]^M$

$$[m^1]^F = rot_z(\pi)tran_y(3)[m^1]^M = \begin{bmatrix} -1 \\ -3 \\ 0 \\ 1 \end{bmatrix}$$

Answer: $[m^1]^F = [-1 \ -3 \ 0 \ 1]^T$

Hold this answer in mind. The next slide runs the same two operations in the opposite order.

Worked example: rotate, then translate

The same two operations, swapped — and a different answer

Rotate M1 about f3 by π, then translate M1 along f2 by 3 units.

$[m^1]^F$ = trany(3) · rotz(π) · $[m^1]^M$

$$[m^1]^F = tran_y(3)rot_z(\pi)[m^1]^M = \begin{bmatrix} -1 \\ 3 \\ 0 \\ 1 \end{bmatrix}$$

Answer: $[m^1]^F$ = [−1 3 0 1]T

Compare with the previous slide: [−1 −3 0 1]T. The rotation is the same, so x is the same; only the y translation is affected by the order.

Two operations, two orders, two different points in space. This is the whole reason the algorithm specifies which side to multiply on.

Inverse transformation

CORE

Reversing a pose costs a transpose and a matrix–vector product

$$ {}^{A}T_{B} = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \quad \Longrightarrow \quad {}^{B}T_{A} = ({}^{A}T_{B})^{-1} = \begin{bmatrix} R^{T} & -R^{T}p \\ 0 & 1 \end{bmatrix} $$

Proof:

$$ TT^{-1} = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R^{T} & -R^{T}p \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} RR^{T} & -RR^{T}p + p \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} I & 0 \\ 0 & 1 \end{bmatrix} = I $$

RRT = I collapses the rotation block, and −RRTp + p = 0 collapses the translation block. The product is the identity.

SE(3): the modern name for the 4×4

OPTIONAL

Rigid-body pose in 3D — rotation and translation together

The notation you will meet in every modern robotics text and library

$R \in SO(3)$

Special Orthogonal group

ORIENTATION only

$$3 \times 3 \quad \cdot \quad R^T R = I \quad \cdot \quad \det R = +1$$

3 independent parameters
(9 numbers, 6 constraints)

$T \in SE(3)$

Special Euclidean group

FULL RIGID-BODY POSE

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}$$

3×3 rotation

3×1 position

6 independent parameters

$$SE(3) = \left\{ T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} : R \in SO(3), p \in \mathbb{R}^3 \right\}$$

Screw transformation

CORE

A translation and a rotation about the same axis, taken together

$$screw(\lambda, \phi, k) = rot(\phi, k)trans(\lambda \mathbf{i}^k)$$

$$rot(\phi, k)trans(\lambda \mathbf{i}^k) = trans(\lambda \mathbf{i}^k)rot(\phi, k), \quad 1 \leq k \leq 3$$

Translate M along an axis of F by λ and rotate about that same axis by φ: the k-th fundamental screw.

Any rigid-body motion is a single screw motion (Chasles' theorem).

Included for completeness and as a pointer to further reading. Not required for the project.

Rotation about an arbitrary axis

CORE

Any two orientations differ by a single rotation about one axis

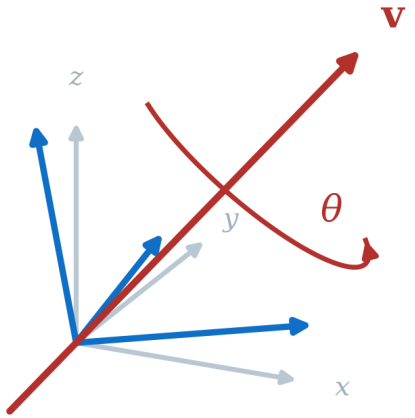
Euler's rotation theorem: any two orientations differ by a single rotation, through an angle θ, about one unit axis v.

It says nothing about position: a statement about R alone.

So orientation has a fourth description: a unit axis and an angle — four numbers, one constraint $\|v\| = 1$, three free parameters as always.

It is also the natural way to interpolate between two orientations — the basis of quaternion interpolation in L5.

```
>> [theta, v] = tr2angvec(R)     % theta in radians
```



Getting the axis and angle out of R

OPTIONAL

MATLAB / Corke Spatial Math: eigenvector with eigenvalue 1

```
R = rotx(0.1)*roty(0.2)*rotz(0.3);
[V,D] = eig(R);
[~,k] = min(abs(diag(D)-1));
v = real(V(:,k));
v = v/norm(v);
s = [R(3,2)-R(2,3);
     R(1,3)-R(3,1);
     R(2,1)-R(1,2)];
if dot(v,s)<0, v=-v; end
c = (trace(R)-1)/2;
theta = acos(max(-1,min(1,c)));
```

One valid eigendecomposition (rounded)

$$V = \begin{bmatrix} 0.6655 & 0.6655 & 0.3379 \\ -0.1220 - 0.6079i & -0.1220 + 0.6079i & 0.4807 \\ -0.2054 + 0.3612i & -0.2054 - 0.3612i & 0.8092 \end{bmatrix}$$

$$D = \begin{bmatrix} 0.9281 + 0.3724i & 0 & 0 \\ 0 & 0.9281 - 0.3724i & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$v \approx \begin{bmatrix} 0.3379 \\ 0.4807 \\ 0.8092 \end{bmatrix}, \quad \theta \approx atan\left(\frac{0.3724}{0.9281}\right) = 21.862^\circ$$

D gives the eigenvalues. Column k of V corresponds to D(k,k).
Eigenvector signs/phases and column order can vary between runs.

Unit quaternions

CORE

Four numbers, no singularity — the representation inside real controllers

```
R = rotx(0.1)*roty(0.2)*rotz(0.3);
q = UnitQuaternion(R);
```

$$q = 0.98186\langle 0.064071, 0.091158, 0.15344 \rangle$$

$$q = s\langle v_1 \quad v_2 \quad v_3 \rangle$$

$$\textit{Rotation by } \theta \textit{ about the unit vector } \mathbf{n}$$

$$s = \cos\frac{\theta}{2}, \quad \mathbf{v} = \sin\frac{\theta}{2}\mathbf{n}$$

A unit quaternion encodes a rotation of θ about a unit axis n.

Axis–angle rearranged, so that composing rotations is a simple product.

q and −q describe the same rotation.

Why quaternions are used in practice

OPTIONAL

Four reasons, all of which matter in Stage 4 of your project

No singularity

Every orientation has a well-defined quaternion. No gimbal lock anywhere in the representation.

Compact and stable

Four numbers instead of nine, with one constraint $\|q\| = 1$. Easy to renormalise after drift.

Smooth interpolation

Spherical linear interpolation gives constant-rate rotation between two orientations — used in L5.

It is what the robot stores

An ABB robtarget carries a quaternion — so in L6 you are writing quaternions whether you realise it or not.

Use quaternions inside code. Use roll–pitch–yaw when a human has to read or type the numbers.

Choosing a representation

CORE

All of them describe the same physical orientation. They are not interchangeable in use.

Representation	Numbers	Unique?	Good for	Watch out for
Rotation matrix R	9	Yes	Computation and composition	6 constraints
Euler angles XYZ moving axes	3	No	Compact orientation description	State sequence and frame Singular at $\beta = \pm\pi/2$
Roll–pitch–yaw XYZ fixed axes	3	No	Operator interfaces and teach pendants	State fixed or moving axes Singular at pitch = $\pm\pi/2$
Axis–angle	4	No	Physical insight and interpolation	Unit axis constraint Axis arbitrary at zero angle
Unit quaternion	4	Twofold	Smooth interpolation without gimbal lock	Unit norm q and −q give the same R

The physical orientation is unique. Its parameterisation is not. State which representation you are using, every time.

Conversions in Corke Spatial Math

Representation	RTB10 / Corke Spatial Math
Rotation about one axis	rotx(a), roty(b), rotz(g)
ZYZ Euler angles	eul2r(phi,theta,psi), tr2eul(R)
RPY: Rz(yaw) Ry(pitch) Rx(roll)	rpy2r(roll,pitch,yaw), tr2rpy(R)
Axis and angle	angvec2r(theta,v), [theta,v]=tr2angvec(R)
Pose and translation	T=rt2tr(R,p), [R,p]=tr2rt(T)
Unit quaternion	q=UnitQuaternion(R), q.R, q.T

Angles are in radians. eul2r uses ZYZ, not the course moving-axis XYZ convention.

Checking a transform before you trust it

OPTIONAL

Five checks that take a minute and save an afternoon

$$R^T R \approx I \qquad detR \approx +1 \qquad TT^{-1} \approx I$$

1. Is R a rotation?

RTR = I, and det R = +1.

det R = −1 is a reflection.

2. Round trip gives I?

T · T-1 = I.

Catches a forgotten −RTp.

3. Inner indices cancel?

BTWobj · ˣTO is meaningless unless X is Wobj.

4. Units and convention?

mm or m; rad or deg; fixed or moving axes.

5. One known case right?

Push {O} through the chain by hand.

Use these in L5

One of the five usually explains a disagreement.

PART 3

Frame chains and project application

From workcell frame networks to Stage 1 evidence

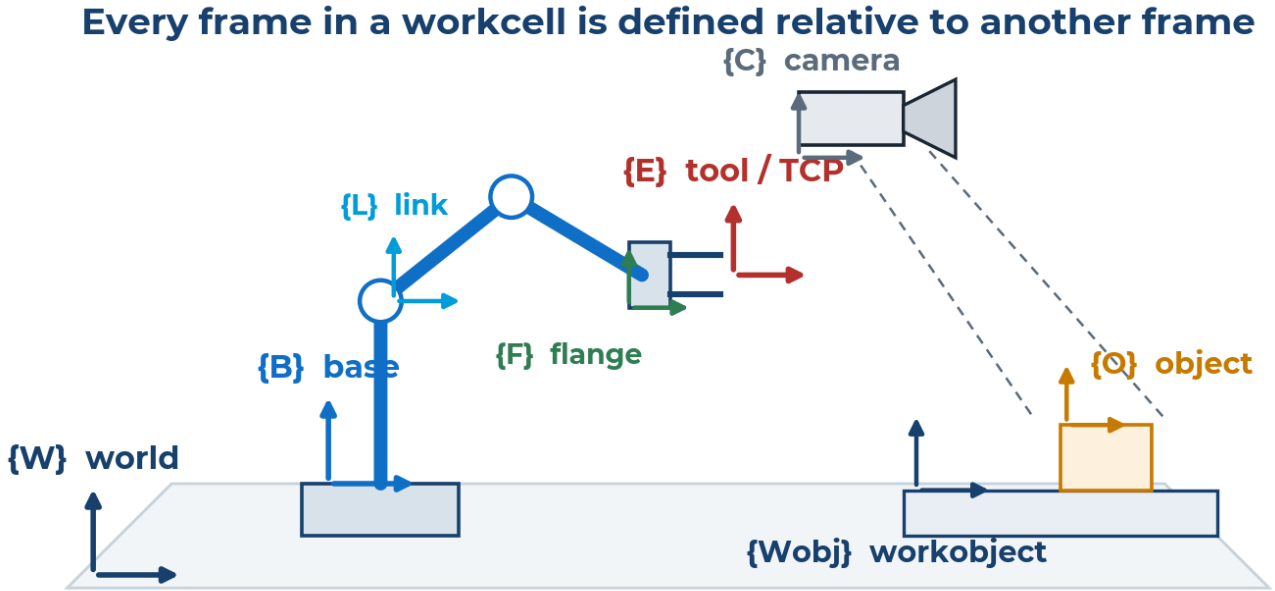
1 Frame trees and transformation chains Slides 43–47
2 Project application Slides 48–51
3 Lecture summary Slide 52

Robotics is largely the study of relationships between coordinate frames.

Not one coordinate system. A connected network of them — and the rules for moving between them.

The frames you will meet in a workcell

CORE

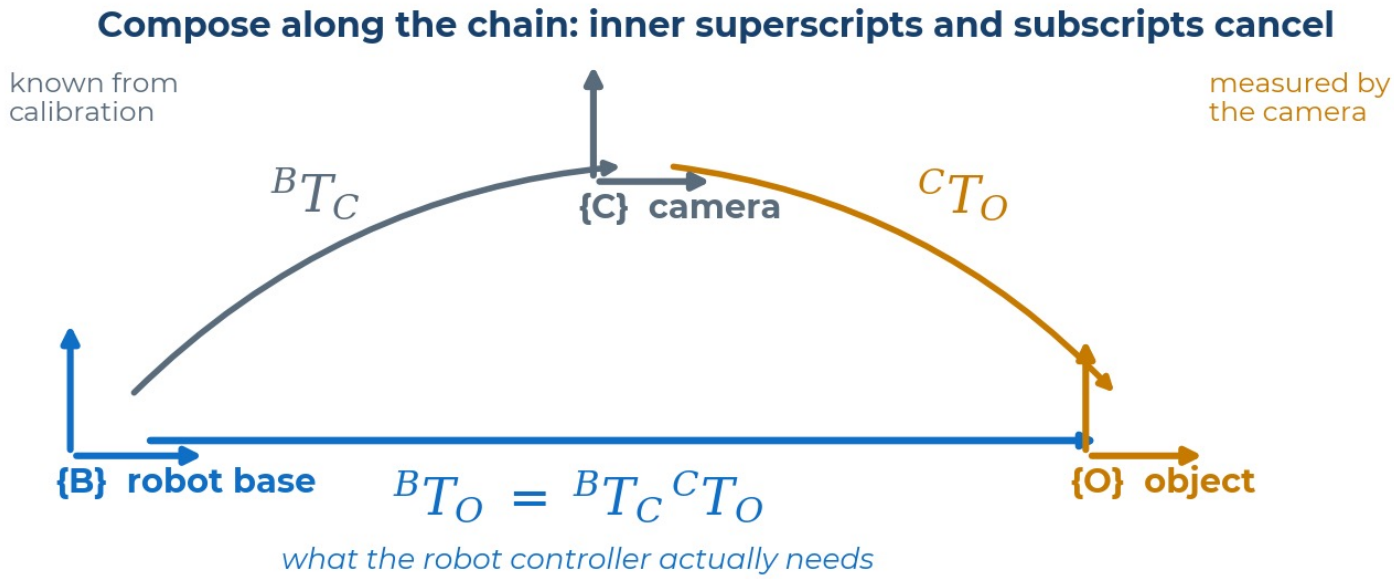


Nothing here has an absolute position — only relationships

These letters are used consistently for the rest of the course: {W} world · {B} base · {F} flange · {E} tool (TCP) · {Wobj} workobject · {O} object · {C} camera.

Composing along a chain

CORE



$$ {}^{A}T_{C} = {}^{A}T_{B}\,{}^{B}T_{C} $$

Read it as a path

Inner indices must match, then cancel: B→C→O collapses to B→O.

The same idea written abstractly: relative pose ξ

OPTIONAL

You will meet this notation in Corke and in the literature — it is what you already know

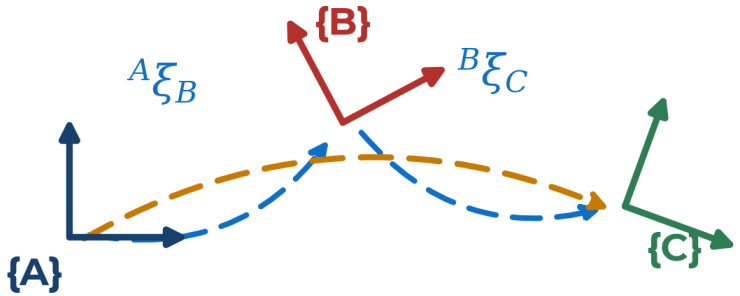
The relative pose of {B} with respect to {A} is written ${}^{A}\xi_{B}$.

With no leading superscript, ξ_B is relative to the world frame {W}.

Read it as an instruction: take {A}, displace and rotate it, and you have {B}. Composition is then:

$${}^{A}\xi_{C} = {}^{A}\xi_{B} \oplus {}^{B}\xi_{C} \qquad {}^{A}\mathbf{P} = {}^{A}\xi_{B} \cdot {}^{B}\mathbf{P}$$

Nothing new: ξ is the transform T, and ⊕ is matrix multiplication.



One symbol, one slide. Recognise it when you read a paper; write T when you do the work.

Frame graphs and loop equations

OPTIONAL

Here B labels the observed object only; robot base frames elsewhere use B.

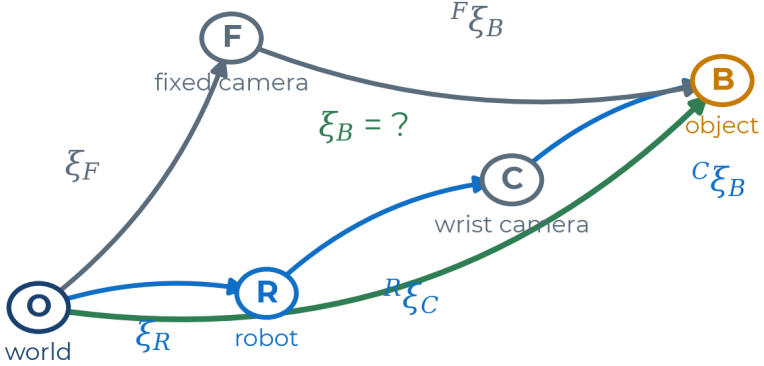
Two routes to the same pose:

via the fixed camera, and via the robot and its wrist-mounted camera.

Both sides start and end at the same node, so they must be equal.

If they are not equal in practice, one of the transforms is wrong — and the size of the mismatch tells you how wrong.

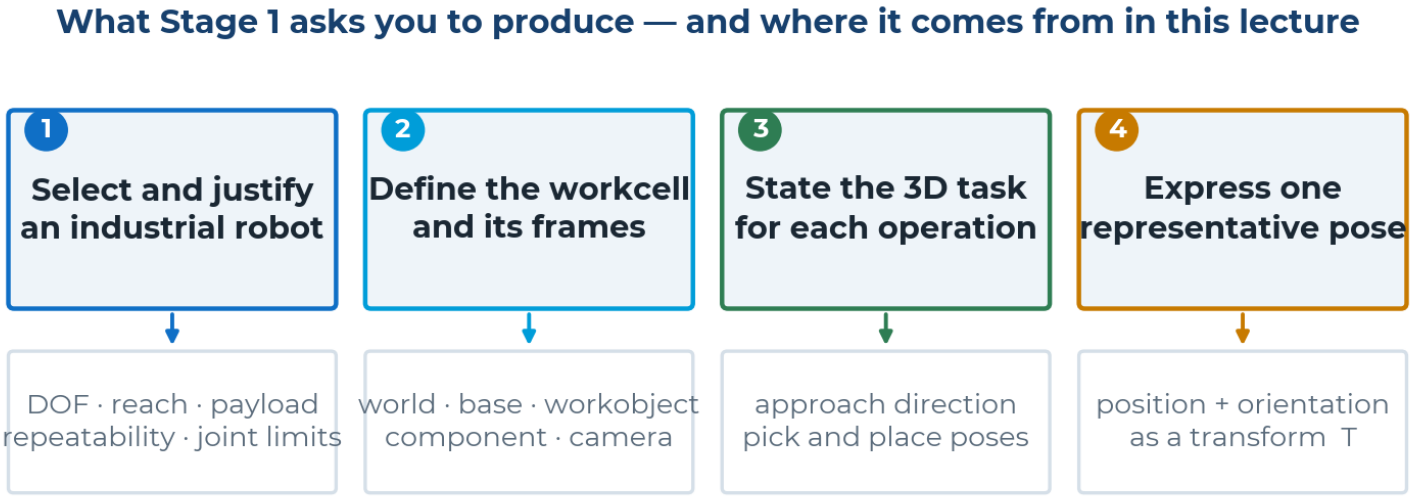
$$\xi_B = \xi_F \oplus {}^F\xi_B = \xi_R \oplus {}^R\xi_C \oplus {}^C\xi_B$$



From this lecture to your tender

TASK

Stage 1 deliverables, and the slide in this lecture that supplies each one



All four are due at the EOI review — one slide, 5 marks

Robot selection, reach and payload come from L01. What this lecture adds is the ability to state your targets as poses — so the reach argument can be checked.

Worked frame example (1): the scene

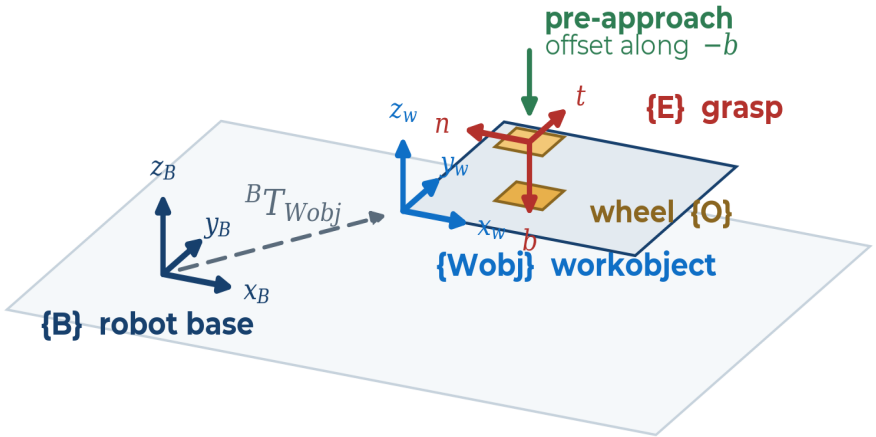
EXAMPLE

Wheel pick in a y-jaw teaching frame

Given for this separate teaching example:

$$ {}^{B}T_{Wobj} = \begin{bmatrix} 1 & 0 & 0 & 650 \\ 0 & 1 & 0 & 200 \\ 0 & 0 & 1 & 150 \\ 0 & 0 & 0 & 1 \end{bmatrix} mm $$

$$ {}^{Wobj}p_O = (120,80,25)\ mm $$



And from the tool and the task:

— the wheel is picked from directly above, so b = (0, 0, −1)' in {B};

— the jaws close along y, so t = (0, 1, 0)', and n = t × b = (−1, 0, 0)';

— the pre-approach standoff is 80 mm.

Worked frame example (2): the calculation

EXAMPLE

Step 1 — put the wheel position into base coordinates

$$ {}^{B}p_O = {}^{B}T_{Wobj} \begin{bmatrix} {}^{Wobj}p_O \\ 1 \end{bmatrix} = (770,280,175)\ mm $$

Step 2 — orientation

$$ {}^{B}R_E = [\mathbf{n}\ \mathbf{t}\ \mathbf{b}] = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & -1 \end{bmatrix} $$

Step 3 — assemble the 4×4 pick pose

$$ {}^{B}T_E = \begin{bmatrix} -1 & 0 & 0 & 770 \\ 0 & 1 & 0 & 280 \\ 0 & 0 & -1 & 175 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

This is goal pose for wheel pick up

Step 4 — back off 80 mm along the approach direction, in the TOOL frame

$$ {}^{B}T_{pre} = {}^{B}T_E Trans(0,0,-80) = \begin{bmatrix} -1 & 0 & 0 & 770 \\ 0 & 1 & 0 & 280 \\ 0 & 0 & -1 & 255 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

This is approach pose for wheel pick up

Worked frame example (3):

EXAMPLE

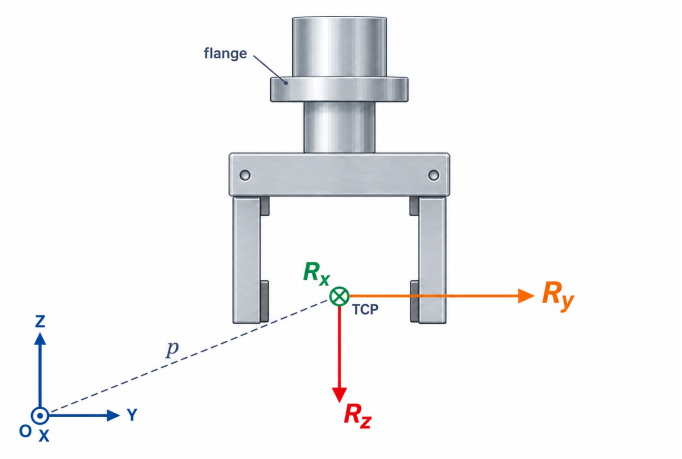
What the four numbers mean, and how you would check them

What you now have

Two robot targets:

pick (770, 280, 175) mm
approach-pick (770, 280, 255) mm

Both use the y-jaw teaching frame. Your project tool frame is your design choice.



For your project, derive target poses from Figure 2 and your chosen gripper and TCP.

The eight things to take away

CORE

From a physical workcell to numbers a controller can act on

1. Robotics is relationships between frames.
2. Pose is position and orientation, always relative to a named frame.
3. The columns of R are the tool axes expressed in the reference frame.
4. R has nine entries but only three independent parameters.
5. R-1 = RT. Inverting an orientation costs nothing but a transpose.
6. **Fixed axis → pre-multiply. Moving axis → post-multiply.**
7. **Order matters: transformation matrices do not commute.**
8. Orientation is unique; its parameterisation is not.

A physical machine became a set of frames; the frames became matrices; the matrices became a chain you can compute with.


UNIVERSITY
OF WOLLONGONG
AUSTRALIA
Stand
for purpose
THANK YOU!